

Vítejte u třetího projektu do SUI! V tomto projektu si procvičíte trénování jednoduchých neuronových sítí. Dost jednoduchých na to, abyste pro výpočty nepotřebovali grafickou kartu. Na druhé straně, dost složitých na to, abychom Vás již netrápili implementací v holém NumPy. Vaším nultým úkolem bude nainstalovat si PyTorch, na [domovské stránce projektu](#) si můžete nechat vygenerovat instalační příkaz pro Vaše potřeby.

Odevzdejte prosím dvojici souborů: Vyrenderované PDF a vyexportovaný Python (File -> Download as). Obojí **pojmenujte loginem vedoucího týmu**. U PDF si pohlídejte, že Vám nemizí kód za okrajem stránky.

V jednotlivých buňkách s úkoly (což nejsou všechny) nahrazujte `pass` a `None` vlastním kódem.

V průběhu řešení se vždy vyvarujte cyklení po jednotlivých datech.

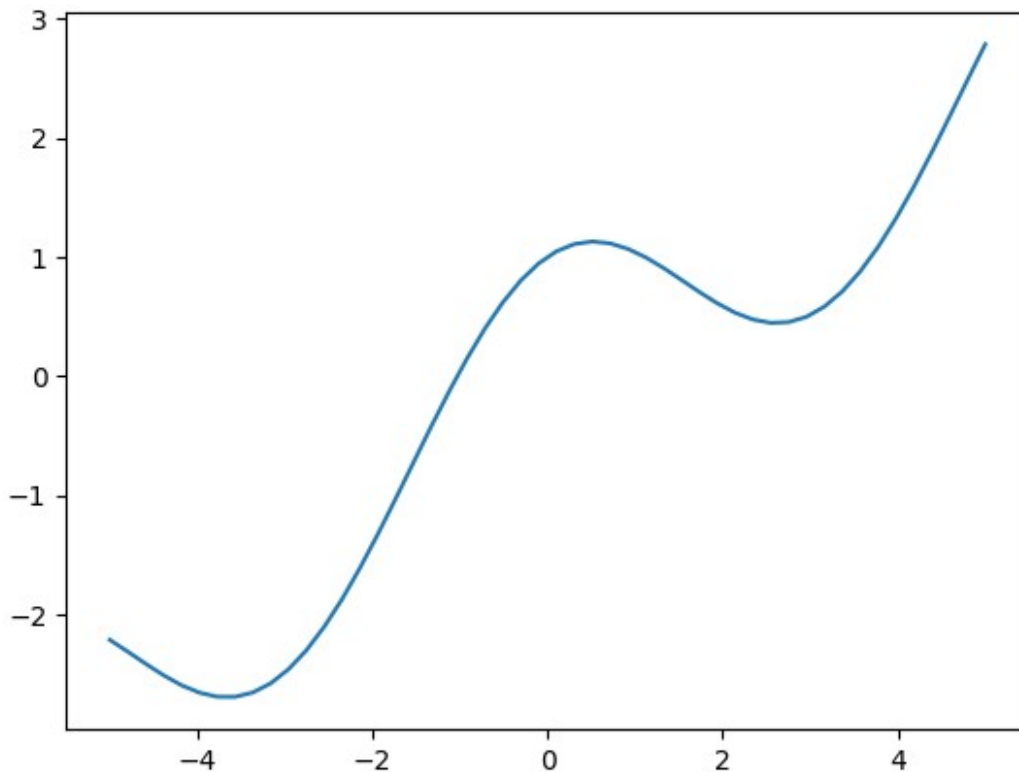
```
import torch
import numpy as np
import matplotlib.pyplot as plt
```

Celý tento projekt bude věnován regresi, tj. odhadu spojitě výstupní veličiny. V první části projektu budete pracovat s následující funkcí:

```
def func(x):
    return torch.cos(x) + x/2

xs = np.linspace(-5, 5, 50)

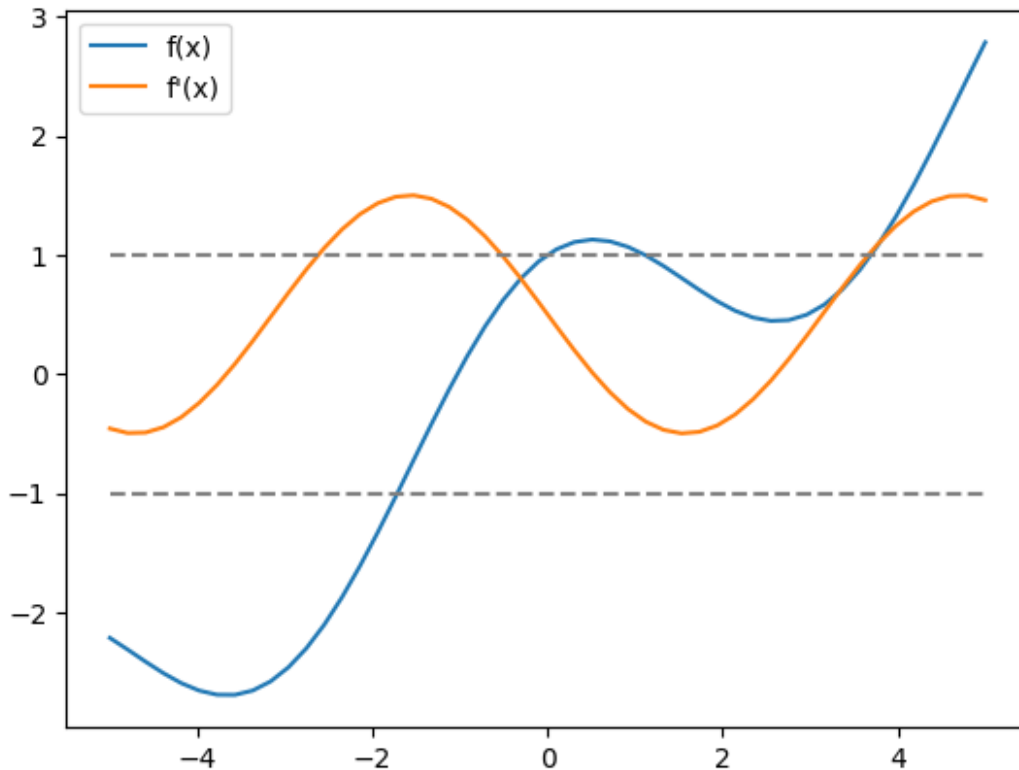
plt.plot(xs, func(torch.tensor(xs)))
plt.show()
```



Vaším prvním úkolem bude pomocí PyTorchu vypočítat hodnoty derivace této funkce na rozsahu $<-5, 5>$. Vytvořte si tensor x ů a řekněte PyTorchu, že budete vzhledem k němu chtít spočítat gradienty (defaultně se to u `Tensoru` nepředpokládá). Pomocí back-propagace je pak vypočítejte. PyTorch umí backpropagovat jenom skalár, najděte tedy způsob, jak agregovat všechny výstupy funkce tak, aby složky gradientu agregované hodnoty byly hodnotami derivace funkce `func` v jednotlivých `x`ech.

```
xs = torch.tensor(np.linspace(-5, 5, 50), requires_grad=True)
fs = func(xs)
fs.sum().backward()

plt.plot(xs.detach(), fs.detach(), label="f(x)")
plt.plot(xs.detach(), xs.grad, label="f'(x)")
plt.plot(xs.detach(), 1 * np.ones(xs.shape[0]), color='gray',
         linestyle='--')
plt.plot(xs.detach(), -1 * np.ones(xs.shape[0]), color='gray',
         linestyle='--')
plt.legend(loc="upper left")
plt.show()
```



Dále budete hledat lokální minimum této funkce. Naimplementujte funkci `tangent_minimum`, která -- v blízké podobnosti metodě tečen -- nalezne řešení, resp. vrátí posloupnost jednotlivých bodů, jimiž při hledání minima prošla. Jejími vstupy jsou:

- `function` -- PyTorch-kompatibilní funkce
- `x0` -- počáteční bod
- `nb_steps` -- zadaný počet kroků, který má být proveden. Ve výstupu tedy bude `nb_steps + 1` položek (vč. `x0`)

Reálně implementujte gradient descent, tedy iterativně vypočítejte hodnotu gradientu (derivace) v aktuálním bodě řešení a odečtěte ji od onoho bodu. Neuvažujte žádnou learning rate (resp. rovnou jedné) a nepoužívejte žádné vestavěné optimalizátory z PyTorch.

Zbýlý kód v buňce pak funkci zavolá a vykreslí, jak postupovala.

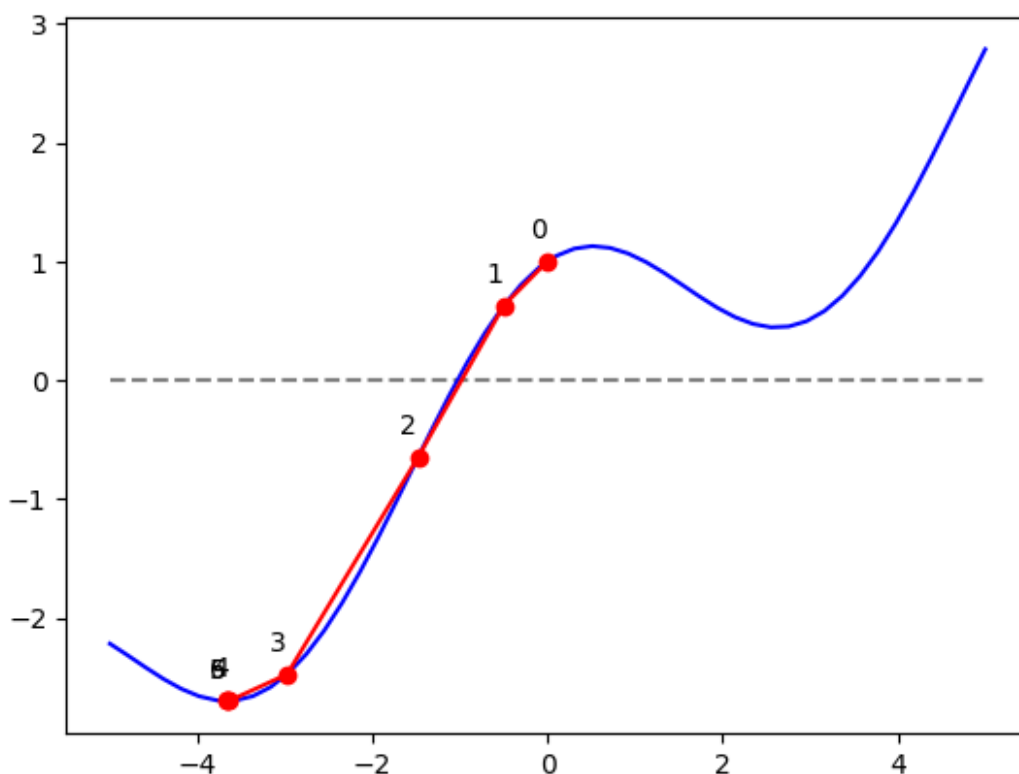
```
def tangent_minimum(function, x0, nb_steps):
    x0.clone().requires_grad_(True)
    updates = [x0.item()]
    for _ in range(nb_steps):
        x0.data -= torch.autograd.grad(function(x0), x0)[0]
        updates.append(x0.item())
    return updates

x0 = torch.tensor([0.0], requires_grad=True)
updates = tangent_minimum(func, x0, 6)
```

```
plt.figure()
plt.plot(xs.detach(), 0 * np.ones(xs.shape[0]), color='gray',
         linestyle='--')
plt.plot(xs.detach(), func(xs).detach(), 'b')
plt.plot(updates, func(torch.tensor(updates)).detach(), 'r',
         marker='o')

for i, (x, y) in enumerate(zip(updates,
                               func(torch.tensor(updates)).detach())):
    plt.annotate(f'{i}', (x, y), xytext=(x-0.2, y+0.2))

plt.show()
```



Modelování polynomů

V následujících několika buňkách budete usilovat o modelování této křivky pomocí polynomů. Prvním krokem bude implementace třídy `LinearRegression`, která bude implementovat ... lineární regresi, pomocí jediného objektu třídy... `torch.nn.Linear`! Po vytvoření objektu `torch.nn.Linear` sáhněte do jeho útrob a nastavte na nulu bias a všechny váhy kromě nulté -- tu nastavte na jednu polovinu. Tím získáte model $y = \frac{x}{2}$, který pro nadcházející úlohu není úplně mimo, a nebudete se tak trápit s dramatickým dynamickým rozsahem loss.

Nechť `LinearRegression` dědí od `torch.nn.Module`, výpočet tedy specifikujte v metodě `forward()`. Při výpočtu zařídte, aby byl výstup ve tvaru `[N]`, nikoliv `[N, 1]`; zároveň to ale

nepřežehňte a pro jediný vstup vracejte stále vektor o rozměru `[1]` a ne jen skalár. Dále naimplementujte metodu `l2_norm()`, která vrací eukleidovskou velikost všech parametrů modelu dohromady, jakoby tvořily jediný vektor. Může se vám hodit `torch.nn.Module.parameters()`.

```
import torch
import torch.nn as nn

class LinearRegression(nn.Module):
    def __init__(self, input_dim):
        super().__init__()

        self.linear = nn.Linear(input_dim, 1, bias=False)
        with torch.no_grad():
            self.linear.weight.fill_(0.5)

    def forward(self, x):
        return self.linear(x).squeeze(1)

    def l2_norm(self):
        return torch.norm(torch.cat([p.view(-1) for p in
self.parameters()]))
```

Naimplementujte funkci pro trénování modelu takového modelu. Funkce přijímá:

- `model` -- PyTorch-kompatibilní model
- `loss_fun` -- funkci, která konzumuje výstupy modelu a cílové hodnoty a model (kvůli regularizaci)
- `optimizer` -- PyTorch-kompatibilní optimalizátor
- `train_X` -- trénovací data ve formátu `[N, F]`
- `train_t` -- cílové hodnoty ve formátu `[N]`
- `nb_steps` -- počet kroků, které se mají provést

Funkce potom vrací průběh trénovací MSE a průběh velikosti parametrů (předpokládejte, že `model` poskytuje `.l2_norm()`). Tedy, dodaná `loss_fun` je použita pouze pro optimalizaci, ale nikde se její hodnoty nelogují.

Dále naimplementujte třídu `MSE_with_regression`, jejíž instance budou sloužit jako mean-square-error loss, navíc rozšířená o L2 regularizaci, jejíž sílu určí uživatel při konstrukci parametrem `l2_beta`.

```
import torch

def train_regression_model(model, loss_fun, optimizer, train_X,
train_t, nb_steps=100):
    mses = []
    norms = []

    for _ in range(nb_steps):
```

```

        output = model(train_X)
        loss = loss_fun(output, train_t, model)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        mses.append(loss.item())
        norms.append(model.l2_norm().item())

    return mses, norms

class MSE_with_regression:
    def __init__(self, l2_beta=0.0):
        self.loss = torch.nn.MSELoss()
        self.l2_beta = l2_beta

    def __call__(self, y, t, model):
        mse = self.loss(y, t)
        l2 = 0.5 * self.l2_beta * model.l2_norm()**2
        return mse + l2

```

Spusťte trénování několikrát pomocí `try_beta` a najděte tři nastavení, která dají po řadě:

1. Dobrý odhad.
2. Silně potlačený odhad regrese, kde ale bude pořád dobře zřetelný trend růstu
3. Extrémně zregularizovaný model, který de facto predikuje konstantu.

Omezte se na interval $\langle 1e-10, 1e+10 \rangle$.

```

def plot_training_result(model, losses, norms):
    fig, axs = plt.subplots(ncols=2, figsize=(13, 6))
    axs[0].plot(xs.detach(), model(xs.float().unsqueeze(-1)).detach())
    axs[0].scatter(data, ts, c='r')

    axs[1].plot(losses, 'r-', label='mse')
    axs[1].legend(loc="upper left")
    axs[1].set_ylim(bottom=0)
    ax_2 = axs[1].twinx()
    ax_2.plot(norms, 'b-', label='norms')
    ax_2.legend(loc="upper right")
    ax_2.set_ylim(bottom=0)

xs = torch.linspace(-5, 5, steps=100)
data = torch.tensor([-4.99, 3.95, -1.5, -0.15, 0, 0.15, 2,
4.9]).unsqueeze(-1)
ts = func(data).squeeze(-1).detach()

def try_beta(l2_beta):

```

```

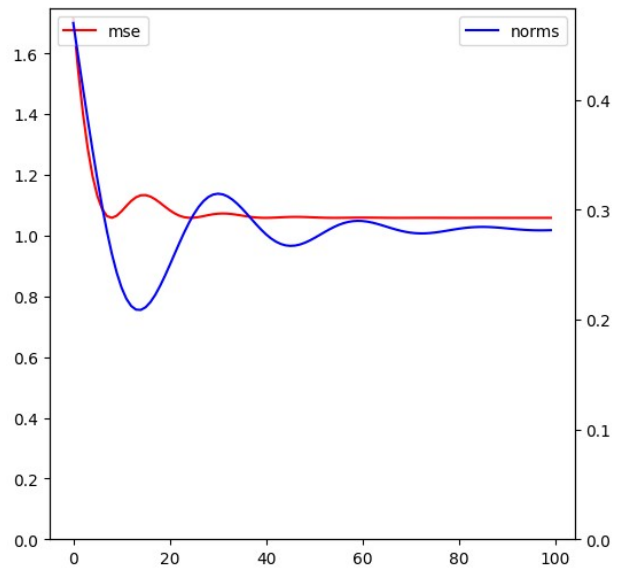
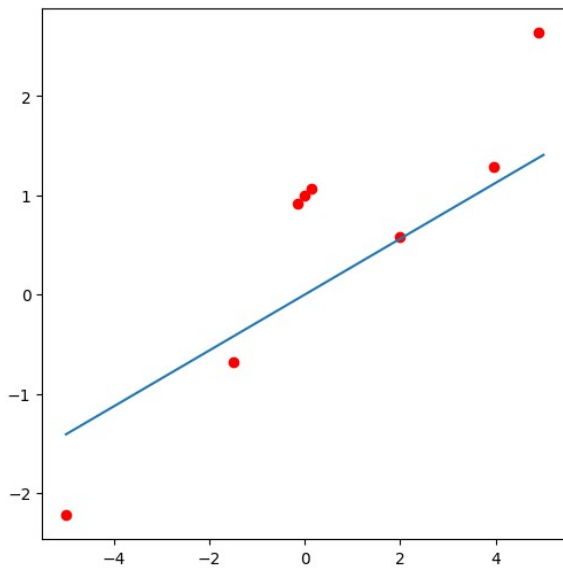
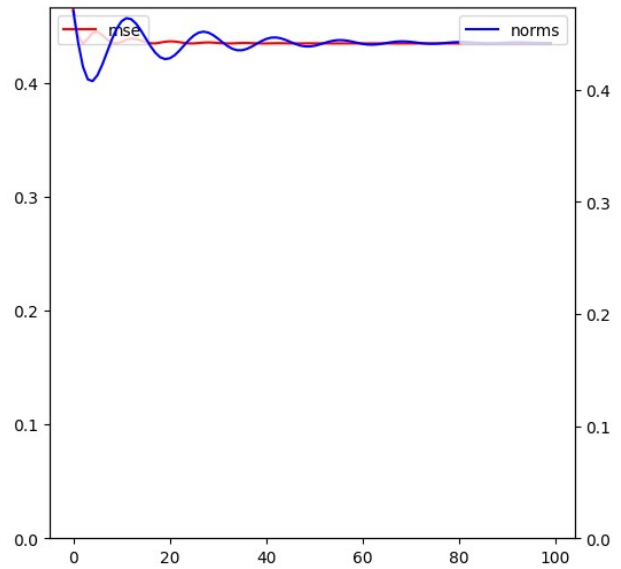
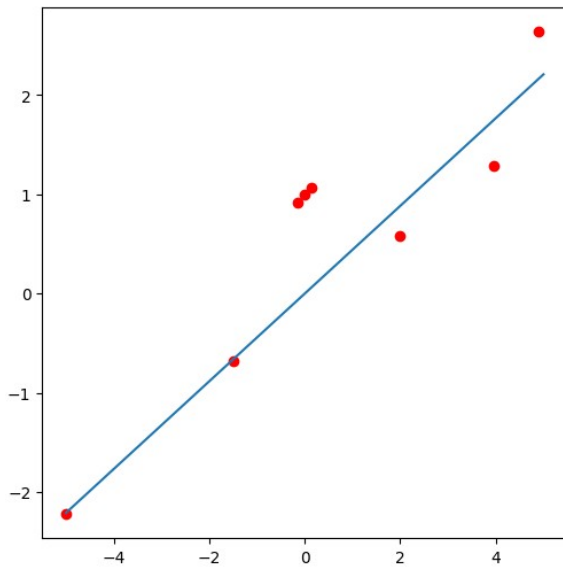
regr_1 = LinearRegression(1)
opt = torch.optim.Adam(regr_1.parameters(), 3e-2)
losses, norms = train_regression_model(regr_1,
MSE_with_regression(l2_beta), opt, data, ts)
plot_training_result(regr_1, losses, norms)

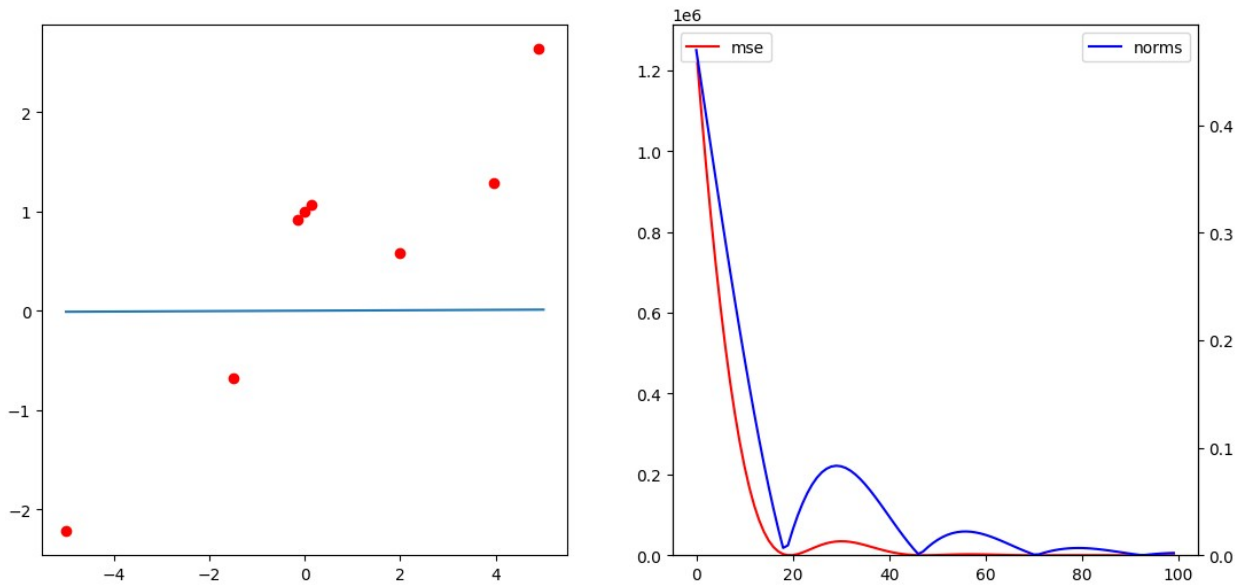
```

```

try_beta(1e-5)
try_beta(1e+1)
try_beta(1e+7)

```





Zde doimplementujte metodu forward pro `PolynomialRegression`. Je potřeba vytvořit rozšířené příznaky a slepit je do jednoho tensoru o tvaru `[N, F]`, který předložíte `self.lin_reg`. Nezapomeňte pak výstup opět omezit na `[N]`.

Zbytek buňky Vám model natrénuje v několika různých variantách řádu polynomu a síly regularizace.

```
class PolynomialRegression1D(torch.nn.Module):
    def __init__(self, order):
        super().__init__()
        self.order = order
        self.lin_reg = LinearRegression(order)

    def forward(self, x):
        features = torch.cat([x ** i for i in range(1, self.order + 1)], dim=1)
        return self.lin_reg(features)

    def l2_norm(self):
        return self.lin_reg.l2_norm()

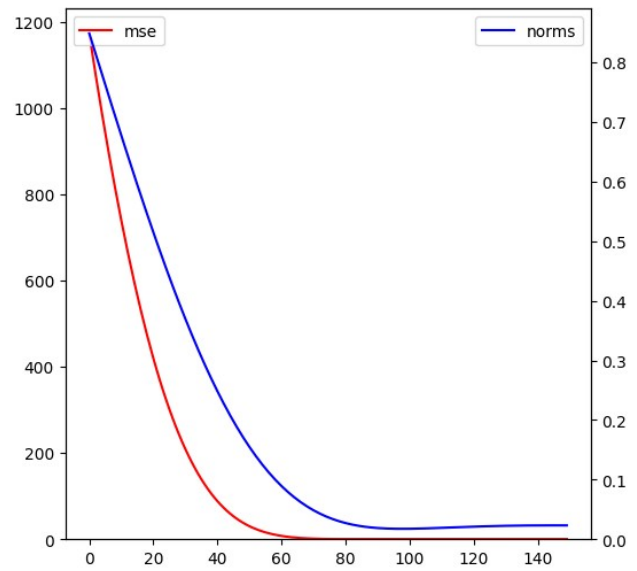
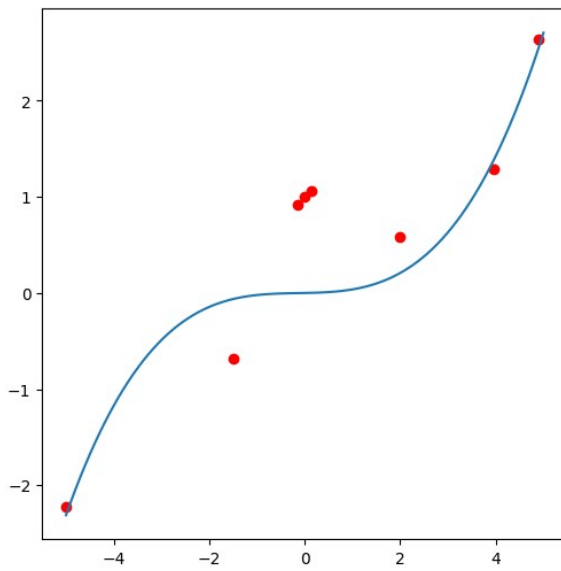
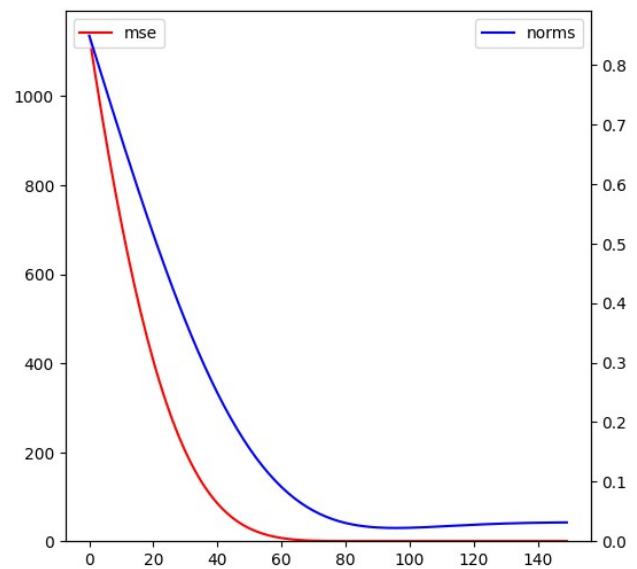
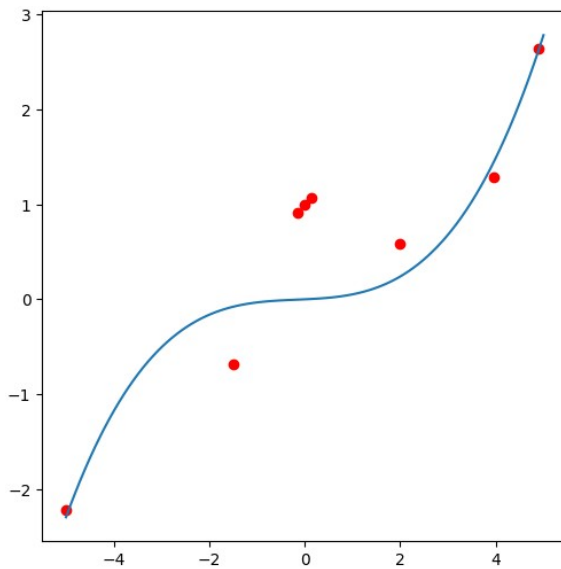
def run_polynomial_regr(order, l2_beta):
    model = PolynomialRegression1D(order)

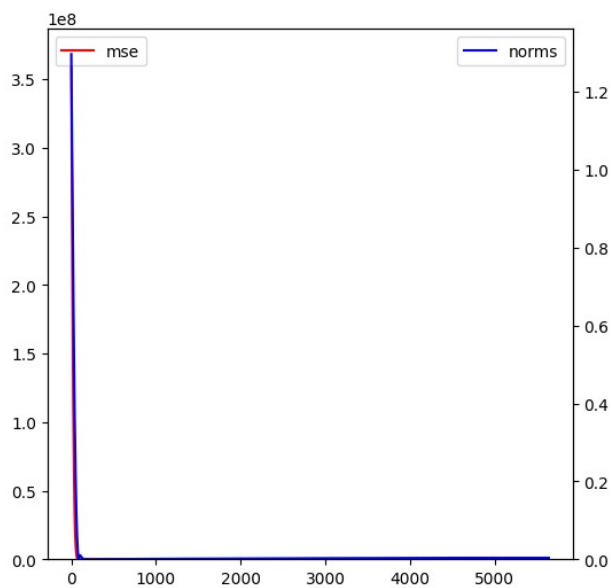
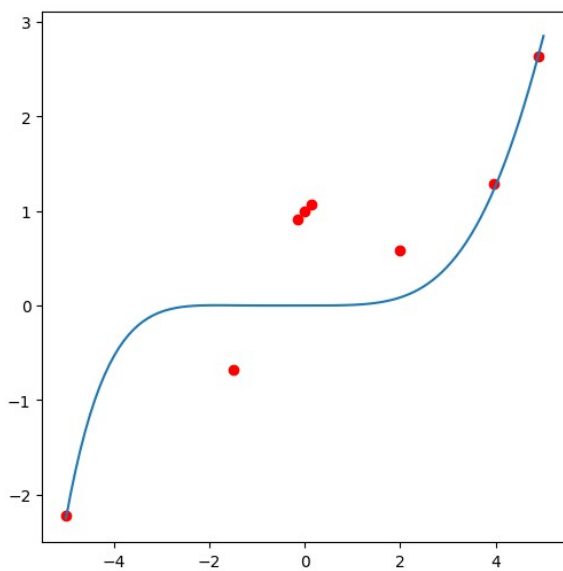
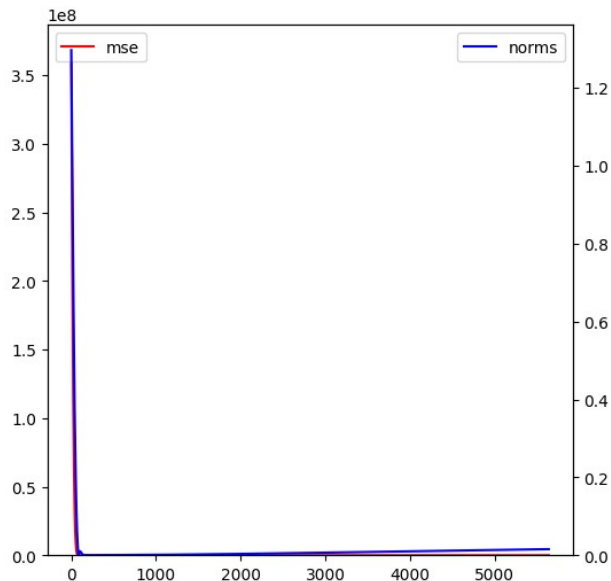
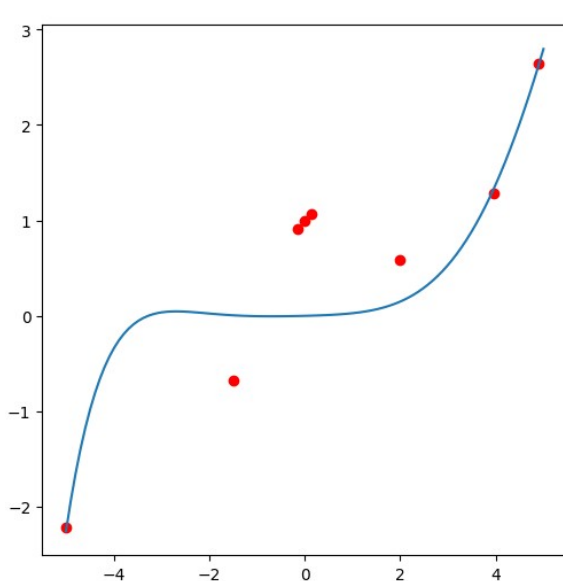
    losses, norms = train_regression_model(
        model,
        MSE_with_regression(l2_beta),
        torch.optim.Adam(model.parameters(), 1e-2),
        data,
        ts,
        nb_steps= 50 + int(100*(order-2)**2.5)
    )
```



```
plot_training_result(model, losses, norms)
```

```
run_polynomial_regr(3, 1e-3)  
run_polynomial_regr(3, 1e+2)  
run_polynomial_regr(7, 1e-1)  
run_polynomial_regr(7, 1e+3)
```





Regrese meteorologických dat

V této části budete usilovat o doplnění tlaku vzduchu z dalších meteorologických měření. Nejprve pomocí lineární regrese, následně pomocí jednoduché neuronové sítě. Každopádně více pomocí vestavěných věcí z PyTorch.

```
turany = np.loadtxt('data-chmu/turany.txt', dtype=np.float32)
mosnov = np.loadtxt('data-chmu/mosnov.txt', dtype=np.float32)
kosetice = np.loadtxt('data-chmu/kosetice.txt', dtype=np.float32)
ruzyne = np.loadtxt('data-chmu/ruzyne.txt', dtype=np.float32)
pribyslav = np.loadtxt('data-chmu/pribyslav.txt', dtype=np.float32)
```

```
features = ['teplota průměrná', 'teplota maximální', 'teplota minimální', 'rychlost větru', 'tlak vzduchu', 'vlhkost vzduchu', 'úhrn srážek', 'celková výška sněhu', 'sluneční svit']
```

V prvním kroce doplňte definici `MeteoDataset` o `__getitem__()` a `__len__()`, tak jak se to očekává u objektů třídy `torch.utils.data.Dataset`. Navíc přidejte vlastnost (`@property`) `in_dim`, která říká, kolik příznaků má každé jedno dato v datasetu.

```
class MeteoDataset(torch.utils.data.Dataset):
    def __init__(self, data, target_feature):
        self.ts = data[target_feature]
        self.xs = data[[i for i in range(data.shape[0]) if i !=
target_feature]].T

    def __getitem__(self, idx):
        x = torch.from_numpy(self.xs[idx]).float().squeeze()
        y =
torch.from_numpy(np.array([self.ts[idx]]).float().squeeze())
        return x, y

    def __len__(self):
        return len(self.ts)

    @property
    def in_dim(self):
        return self.xs.shape[1]

target_feature = 'tlak vzduchu'
train_dataset = MeteoDataset(np.concatenate([mosnov, kosetice,
pribyslav], axis=1), features.index(target_feature))
valid_dataset = MeteoDataset(ruzyne, features.index(target_feature))
test_dataset = MeteoDataset(ruzyne, features.index(target_feature))
print(valid_dataset.xs.shape, valid_dataset.ts.shape)

valid_loader = torch.utils.data.DataLoader(valid_dataset,
batch_size=128, shuffle=False, drop_last=False)
print(len(valid_loader))

(22280, 8) (22280,)
175
```

Zde je definována funkce pro evaluaci modelu. Budete ji používat, ale implementovat v ní nic nemusíte.

```
def evaluate(model, data_loader):
    model.eval()
    total_squared_error = 0.0
    nb_datos = 0
```

```

        with torch.no_grad():
            for X, t in data_loader:
                y = model(X)
                total_squared_error += torch.nn.functional.mse_loss(y, t,
reduction='sum')
                nb_datos += len(t)

        return total_squared_error / nb_datos

evaluate(LinearRegression(train_dataset.in_dim), valid_loader)

tensor(838856.1250)

```

Nad trénovacím dataset vytvořte `DataLoader`, který bude vytvářet minibatche o velikosti 32 příkladů. Poté z něj vytvořte nekonečný proud dat. Můžete k tomu naimplementovat vlastní cyklící iterátor nebo použít vhodnou funkci z `itertools`.

Dále naimplementujte trénovací smyčku ve funkci `train()`, která přijímá:

- `model` -- referenci na model, jenž má být natrénován
- `train_stream` -- iterátor přes trénovací batche
- `optimizer` -- instanci optimalizátoru, který bude využit pro trénování
- `nb_updates` -- počet trénovacích kroků, jež mají být provedeny
- `eval_period` -- po kolika krocích se má vyhodnocovat model na validačních datech
- `valid_loader` -- iterable s validačními daty

Funkce nechť používá `torch.nn.functional.mse_loss()` jako loss. Vracejte průběh validační loss spolu s pořadovými čísly kroků, kdy došlo k měření, tedy jako seznam dvojic `[(i_1, loss_1), ...]`. `model` trénujte přímo.

Zbytek buňky vyzkouší trénování pro několik různých learning rate. Vzhledem k jednoduchosti úlohy jsou to learning rate gigantické oproti prakticky používaným.

```

import itertools

train_loader = torch.utils.data.DataLoader(train_dataset,
batch_size=32, shuffle=True, drop_last=True)
train_stream = itertools.cycle(train_loader)

def train(model, train_stream, optimizer, nb_updates, eval_period,
valid_loader):
    valid_progress = []

    model.train()
    for i in range(1, nb_updates + 1):
        # Získání minibatche z cyklického iterátoru
        X_batch, t_batch = next(train_stream)

        # Výpočet predikce a loss
        y_batch = model(X_batch)

```

```

        loss = torch.nn.functional.mse_loss(y_batch,
t_batch).squeeze()

        # Zpětný průchod a aktualizace vah
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Vyhodnocení na validačních datech v pravidelných intervalech
        if i % eval_period == 0:
            valid_loss = evaluate(model, valid_loader)
            valid_progress.append((i, valid_loss.item()))

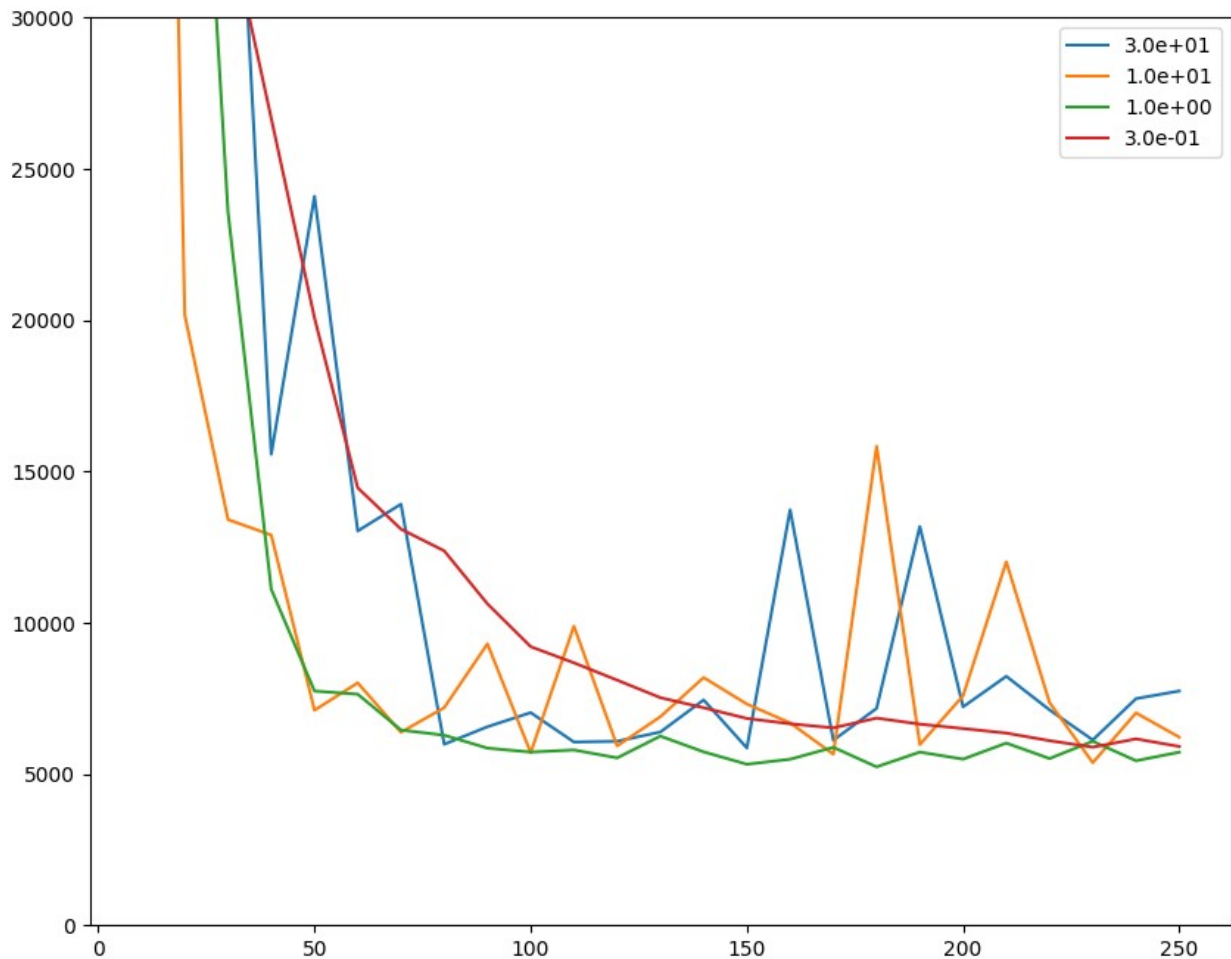
    return valid_progress

def lr_progress(lr):
    linear_predictor = LinearRegression(train_dataset.in_dim)
    optimizer = torch.optim.Adam(linear_predictor.parameters(), lr)
    progress = train(linear_predictor, train_stream, optimizer, 250,
10, valid_loader)
    print(lr, evaluate(linear_predictor, valid_loader))
    return progress

plt.figure(figsize=(10, 8))
for lr in [3e+1, 1e+1, 1e+0, 3e-1]:
    progress = lr_progress(lr)
    plt.plot([item[0] for item in progress], [item[1] for item in
progress], label=f"{lr:.1e}")
plt.legend()
plt.ylim(0, 30000)
plt.show()

30.0 tensor(7743.0679)
10.0 tensor(6216.8691)
1.0 tensor(5720.7124)
0.3 tensor(5910.1196)

```



Konečně naimplementujte jednoduchou neuronovou síť, která bude schopná regrese. Při konstrukci necht' přijímá:

- rozměr vstupu
- počet skrytých vstev
- šířku každé skryté vrstvy
- instanci nonlinearity, která má být aplikována v každé skryté vrstvě

Při dopředném průchodu necht' se uplatní všechny vrstvy, nezapomeňte opět redukovat výstup na [N]. Nejspíš se Vám bude hodit `torch.nn.Sequential`.

Zbytek buňky vyzkouší několik různých konfigurací. Pravděpodobně uvidíte ilustraci faktu, že v rozporu s častou reportovací praxí není počet parametrů nutně tím nejzásadnějším číslem pro odhad síly modelu, tím může být prostě šířka.

```
class LocalMeteoModel(torch.nn.Module):
    def __init__(self, input_dim, nb_layers, layer_width,
nonlinearity):
        super().__init__()
        self.input_dim = input_dim
```

```

    assert nb_layers >= 1

    layers = []
    layers.append(torch.nn.Linear(input_dim, layer_width))
    layers.append(nonlinearity)

    for _ in range(nb_layers - 1):
        layers.append(torch.nn.Linear(layer_width, layer_width))
        layers.append(nonlinearity)

    layers.append(torch.nn.Linear(layer_width, 1))

    self.model = torch.nn.Sequential(*layers)

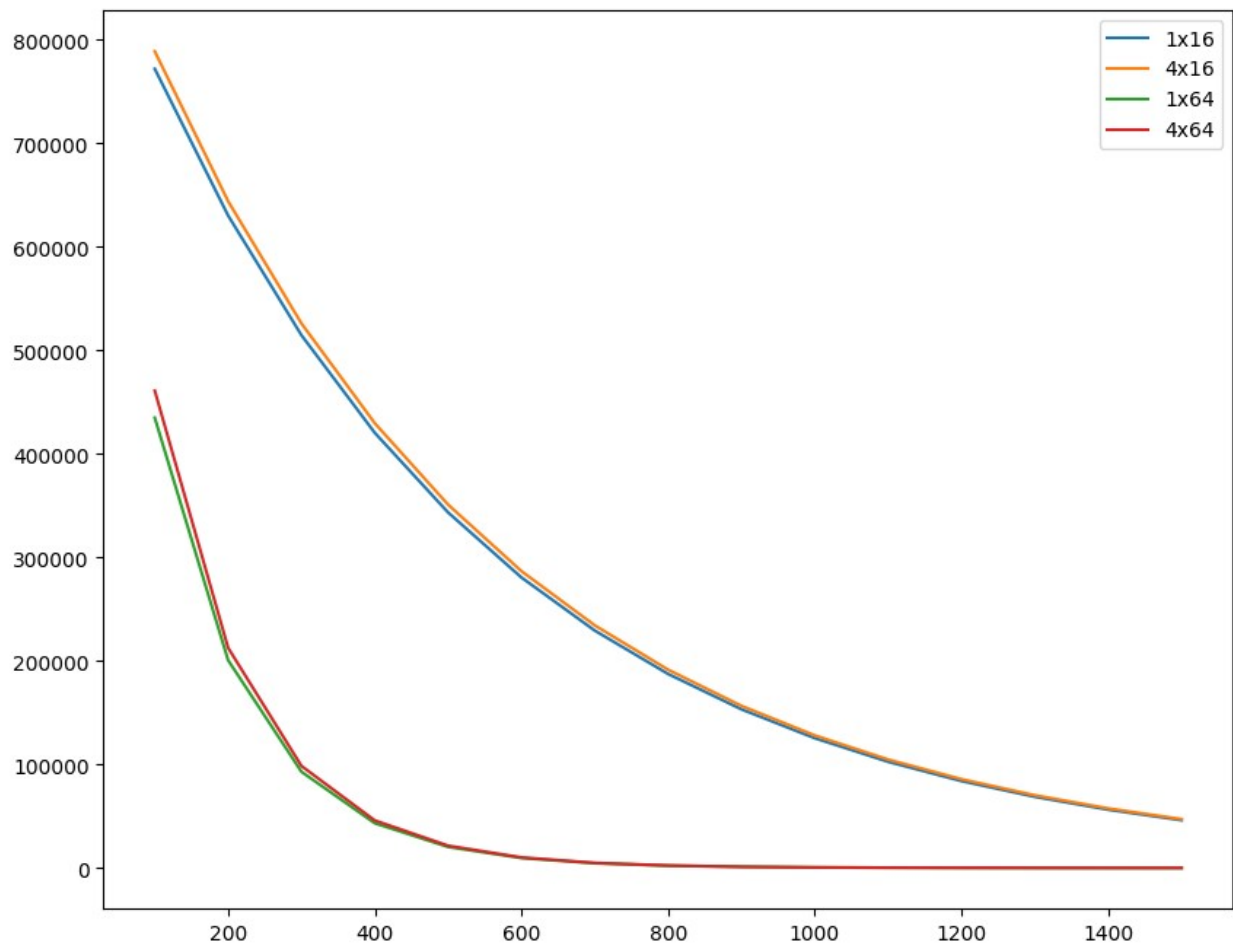
def forward(self, x):
    return self.model(x).squeeze()

def depth_progress(depth, width):
    nn_predictor = LocalMeteoModel(train_dataset.in_dim, depth, width,
    torch.nn.Tanh())
    optimizer = torch.optim.SGD(nn_predictor.parameters(), 3e-5)
    progress = train(nn_predictor, train_stream, optimizer, 1500, 100,
    valid_loader)
    print(f"Depth {depth}, width {width}: {evaluate(nn_predictor,
    valid_loader):.2f}")
    return progress

plt.figure(figsize=(10, 8))
for depth, width in [(1, 16), (4, 16), (1, 64), (4, 64)]:
    progress = depth_progress(depth, width)
    plt.plot([item[0] for item in progress], [item[1] for item in
    progress], label=f"{depth}x{width}")
plt.legend()
plt.show()

Depth 1, width 16: 46309.25
Depth 4, width 16: 47288.74
Depth 1, width 64: 139.79
Depth 4, width 64: 135.76

```



Gratulujeme ke zvládnutí projektu! Při odevzdání nezapomeňte soubory pojmenovat podle vedoucího týmu.